Contexto

Problema real, objetivo e atores da rastreabilidade

EcoOrigem

Bioeconomia amazônica

Problema

Registros ficam separados entre organizações. O consumidor não consegue verificar com facilidade qual histórico permaneceu íntegro e quem executou cada etapa.

Objetivo

Compartilhar um histórico público e verificável de lotes. A MetaMask identifica os participantes, o contrato aplica perfis e estados, e a Sepolia confirma transações e eventos.

Produtor — Beneficiador — Transportador — Distribuidor — Consumidor

A blockchain preserva o registro confirmado, mas não prova que o dado informado na origem era verdadeiro.

Primeira Entrega

A base didática construída antes da integração com a Ethereum

Blockchain local em Python

Blocos encadeados, prova de trabalho, nonce, hash SHA-256 e persistência local.

Integridade verificável

Raiz de Merkle, assinatura de carteiras e validação de transações antes da inclusão no bloco.

Regras de rastreabilidade

Contrato didático com perfis, custódia e ordem de etapas do produtor ao distribuidor.

Aplicação e qualidade

Interface web, API, Docker, logs amigáveis, testes automatizados e relatório técnico.

Resultado: conceito validado em ambiente local

Segunda Entrega

Melhorias e novas implementações para uma dApp pública



Contrato Solidity

Sete grupos obrigatórios usados com regras reais, NatSpec, eventos e erros.



Ethereum Sepolia

Estado público, transações confirmadas e histórico consultável no Etherscan.



MetaMask

Conexão, assinatura de sessão, troca de rede e confirmação de transações.



Frontend Web3.js

Leituras, escritas pagáveis, eventos e erros traduzidos em HTML, CSS e JavaScript.



Documentos e IPFS

SHA-256 e URI on-chain; o arquivo permanece fora da blockchain.



Deploy e testes

Vercel por HTTPS, suíte local, testes Remix e auditoria de credenciais.

IDEs, frameworks e serviços

Cada tecnologia tem uma responsabilidade no ciclo da dApp

Remix IDE

Importa, compila, testa e envia o deploy.

Solidity

Implementa regras, perfis, estados e eventos.

MetaMask

Guarda chaves e solicita assinaturas.

Sepolia

Executa e replica o contrato de teste.

Web3.js 4.x

Liga o frontend à carteira e à ABI.

IPFS

Mantém documentos fora da blockchain.

Vercel

Publica o frontend estático por HTTPS.

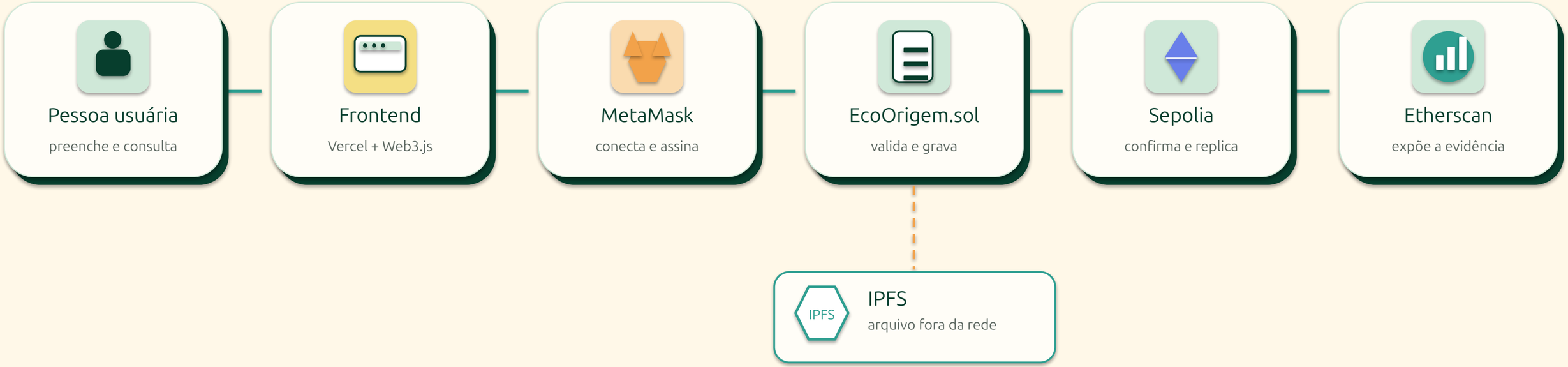
Etherscan

Confere contrato, blocos, eventos e hashes.

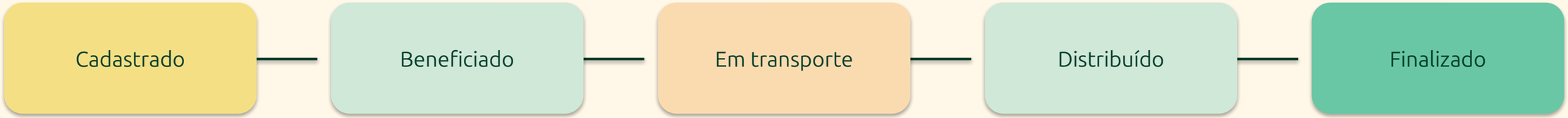
Código → assinatura → rede → evidência

Fluxo completo da dApp

Da ação no navegador até a confirmação pública e o histórico do lote



Máquina de estados do lote



Cada transação válida emite eventos; operações fora do perfil ou da ordem são recusadas sem alterar o estado.

Demonstração ao vivo

O que será mostrado e em qual ferramenta

O que será demonstrado	Onde será mostrado
01 Conectar e autenticar a carteira	Vercel + MetaMask
02 Ler totais, perfil e lotes com call	Frontend
03 Cadastrar lote com taxa em SepoliaETH	Frontend + MetaMask
04 Acompanhar hash, bloco e confirmação	Frontend + Etherscan
05 Provocar erro de perfil e conferir evento	Frontend + Etherscan

Plano B: vídeo de 3 a 5 minutos e capturas reais, sem fazer novo deploy durante a apresentação.

Bônus: relatório de desenvolvimento

Registro técnico preservado como evidência da evolução do projeto

Relatório técnico

Documento de 21 páginas produzido na Primeira Entrega e preservado em PDF e DOCX.

Conteúdo

Arquitetura, decisões de implementação, testes, segurança, resultados e limitações.

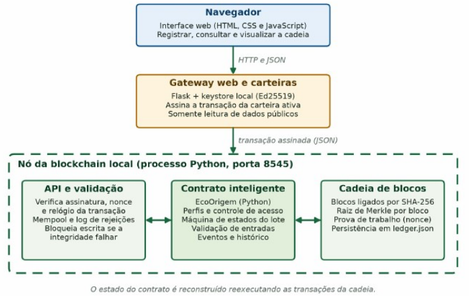
Valor para a Entrega 2

Mostra a base que originou a migração para Solidity, MetaMask, Sepolia, Web3.js e Vercel.

4 Arquitetura

A arquitetura segue o modelo de aplicação descentralizada da Aula 01: interface, contrato inteligente e blockchain, com uma carteira responsável por assinar as transações. A Figura 1 mostra os componentes e a Tabela 2 descreve a responsabilidade de cada um.

Figura 1 - Arquitetura do EcoOrigem



Legenda: o navegador conversa apenas com o gateway web; o gateway assina a transação com a carteira ativa e a envia ao nó, onde o contrato executa as regras e a cadeia registra o resultado.

Tabela 2 - Componentes e responsabilidades

Componente	Tecnologia	Responsabilidade
Interface web	HTML, CSS e JavaScript	Formulários, consulta de lotes, explorador de blocos e laboratório de integridade
Gateway e carteiras	Flask, Ed25519	Guarda as chaves de demonstração, assina a transação e encaminha leituras ao nó
Nó da blockchain	Flask, Python	Recebe transações assinadas, valida, executa o contrato, minera e persiste os blocos
Contrato inteligente	Python (EcoOrigemContract)	Perfis, máquina de estados, validações e eventos
Cadeia de blocos	Python (Block, Blockchain)	Encadeamento por SHA-256, prova de trabalho, raiz de Merkle e validação
Empacotamento	Docker, Makefile, venv	Execução reproduzível, testes em contêiner e menu interativo

Legenda: cada linha corresponde a um pacote do repositório, descrito na seção 5.

Print do relatório

Arquivos: docs/relatorio/relatorio-desenvolvimento-ecoorigem.pdf e .docx

EcoOrigem

Obrigada

Rastreabilidade verificável para a bioeconomia amazônica

Ana Beatriz Maciel Nunes • Fernando Luiz da Silva Freire • Juliana Ballin Lima

github.com/JulianaBallin/EcoOrigem

Oficina de Desenvolvimento de Sistemas III

UEA
UNIVERSIDADE DO ESTADO DO AMAZONAS